A — Context-level learning and memory (the Notebook, reflection, sleep)

0. Role in the system

This section informs the **Notebook** module: a persistent store of facts, lessons and playbook entries that the local coding assistant reads before acting and updates after every task. The module must (a) represent memory as small itemized entries with identifiers, counters and metadata rather than one monolithic prompt; (b) update it only by **delta operations** (ADD / UPDATE counters / link / evolve-neighbor), merged deterministically, never by an LLM full rewrite; (c) split work across a **Generator** (worker 9B), a **Reflector** and a **Curator** (senior 35B-A3B); (d) keep a **small always-in-context core block** with explicit eviction and recursive summarization; (e) run **idle-time consolidation** (reflection, merging, decay, promotion, pre-computation over the repo) when the user is away; (f) turn lessons into executable tests so the verifier cascade can check them. After reading this pack, agents should be able to implement the entry schema, the retrieval scoring function, the per-task reflect-and-curate loop, the grow-and-refine merge, the core-block queue manager and the sleep-time job, with starting hyperparameters traced to a paper.

1. Papers

1.1 Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models (Zhang et al., 2025) — arXiv:2510.04618

Problem and claim. LLM systems increasingly adapt by changing their *inputs* (instructions, strategies, evidence) rather than their weights. The authors identify two failure modes of prior context-adaptation methods: **brevity bias** — “the tendency of optimization to collapse toward short, generic prompts” (Sec. 2.2; documented by Gao et al. for test-generation prompt optimization, where iterative methods kept converging to near-identical instructions like “Create unit tests to ensure methods behave as expected”; GEPA is said to treat brevity as a strength, which omits domain heuristics, tool-use guidance and failure modes) — and **context collapse**, which “arises when an LLM is tasked with fully rewriting the accumulated context at each adaptation step” (Sec. 2.2). Case study (Dynamic Cheatsheet on AppWorld): at step 60 the context held **18,282 tokens** at **66.7** accuracy; at the next step it collapsed to **122 tokens** and **57.1** accuracy — below the **63.7** no-adaptation baseline. ACE’s claim: contexts should be comprehensive, evolving *playbooks* built by three roles with incremental delta updates and grow-and-refine, which avoids both failure modes and scales with long-context models.

Headline results (base LLM DeepSeek-V3.1-671B, non-thinking mode for all three roles). - Agents: **+10.6%** average over strong baselines, up to **+17.1%** over base ReAct (AppWorld). Finance: **+8.6%** average. Adaptation latency reduced **86.9%** on average. Without ground-truth labels, ReAct+ACE offline still gains **+14.8%** average over ReAct. - Leaderboard: ReAct+ACE **59.4%** average vs IBM CUGA **60.3%** (GPT-4.1); under online adaptation ACE surpasses CUGA on test-challenge by **+8.4 TGC** and **+0.7 SGC**.

AppWorld, Table 1 (TGC = Task Goal Completion, SGC = Scenario Goal Completion; GT = ground-truth labels available to the Reflector):

Method	GT	TN-TGC	TN-SGC	TC-TGC	TC-SGC	Avg
ReAct	—	63.7	42.9	41.5	21.6	42.4
ReAct+ICL (offline)	yes	64.3	46.4	46.0	27.3	46.0
ReAct+GEPA (offline)	yes	64.9	44.6	46.0	30.2	46.4
ReAct+ACE (offline)	yes	76.2	64.3	57.3	39.6	59.4
ReAct+ACE (offline)	no	75.0	64.3	54.4	35.2	57.2
ReAct+DC-CU (online)	no	65.5	58.9	52.3	30.8	51.9

Method	GT	TN-TGC	TN-SGC	TC-TGC	TC-SGC	Avg
ReAct+ACE (online)	no	69.6	53.6	66.0	48.9	59.5

Finance, Table 2 (accuracy):

Method	GT	FiNER	Formula	Avg
Base LLM	—	70.7	67.5	69.1
ICL (offline)	yes	72.3	67.0	69.6
MIPROv2 (offline)	yes	72.4	69.5	70.9
GEPA (offline)	yes	73.5	71.5	72.5
ACE (offline)	yes	78.3	85.5	81.9
ACE (offline)	no	71.1	83.0	77.1
DC-CU (online)	yes	74.2	69.5	71.8
DC-CU (online)	no	68.3	62.5	65.4
ACE (online)	yes	76.7	76.5	76.6
ACE (online)	no	67.3	78.5	72.9

Cost and speed, Table 4: offline AppWorld — GEPA **53,898 s / 1,434 rollouts** vs ACE **9,517 s (−82.3%) / 357 rollouts (−75.1%)**; online FiNER — DC-CU **65,104 s / $17.7** vs ACE **5,503 s (−91.5%) / $2.9 (−83.6%)**.

Method — equations. The paper has no equations (Sec. 3 is prose). The only formal object is the bullet format, shown in the Curator example and the released code as

```
[ctx-00263] helpful=1 harmful=0 :: <content>
```

i.e. `[<section-prefix>-<5-digit id>] helpful=<int> harmful=<int> :: <text>`. Prefixes seen: `shr-` (strategies and hard rules), `code-` (code snippets), `ts-` (troubleshooting), `calc-`, `fin-`, `ctx-`. Section headings in the example playbook: STRATEGIES AND HARD RULES; USEFUL CODE SNIPPETS AND TEMPLATES; TROUBLESHOOTING AND PITFALLS. Example bullet (Fig. 3): `ts-00003` — “If authentication fails, troubleshoot systematically: try phone number instead of email as username”.

Method — algorithm (Sec. 3, Fig. 4; prompts in Appendix D). 1. **Generator** solves the task with the playbook inserted between `PLAYBOOK_BEGIN` / `PLAYBOOK_END`, told to read it first and apply relevant sections. Output (FiNER, Fig. 12) is JSON `{reasoning, bullet_ids, final_answer}` where `bullet_ids` are the bullets it actually used. 2. **Reflector** compares the trajectory against execution feedback / test results / ground truth (if any). Output (AppWorld, Fig. 10) JSON `{reasoning, error_identification, root_cause_analysis, correct_approach, key_insight}`; it tags each used bullet as **helpful / harmful / neutral** (`bullet_tags: [{id, tag}]`). May iterate up to a maximum number of refinement rounds. 3. **Curator** turns insights into a **delta context**: JSON `{reasoning, operations: [{type, section, content}]}`. In the released prompts the only `type` is **ADD**; the system assigns ids (the model must not). It is told to avoid redundancy and must not regenerate the playbook. 4. **Deterministic merge** (“lightweight, non-LLM logic”): new-id bullets are appended; existing bullets updated in place (counters incremented from the Reflector’s tags). Because updates are localized, multiple deltas can be merged in parallel (batched adaptation). 5. **Grow-and-refine**: refinement is either *proactive* (after each delta) or *lazy* (only when the context window is exceeded). De-duplication compares bullets by **semantic embeddings**; pruning is described only as part of de-duplication. The released code exposes `--bulletpoint_analyzer_threshold` (cosine, default **0.9**, analyzer off by default), `curator_frequency` (default 1) and `playbook_token_budget` (default **80,000**). 6. **Multi-epoch**: the same training queries are revisited to strengthen the context. Offline = adapt on a train split, freeze, evaluate; online = adapt on the test stream as it arrives, optionally after an *offline warmup*.

Hyperparameters and setup. Batch size 1 (one delta per sample); max Reflector refinement rounds **5**; max offline epochs **5**; pass@1 for offline evaluation; DC baseline = DC-CU (cumulative) official implementation; GEPA and MIPROv2 as offline baselines; official AppWorld protocol and original train/val/test splits (split sizes not in fetched text — verify in paper §4.1).

Ablations that matter (Table 3, AppWorld, GT labels for offline rows): w/o Reflector or multi-epoch **55.1** avg (+12.7 over ReAct); w/o multi-epoch **56.8**; full offline ACE **59.4**. Online ACE w/o warmup **56.1** (67.9/51.8/61.4/43.2); online ACE **+ offline warmup 59.5** (69.6/53.6/66.0/48.9). So: Reflector ≈ +1.7, multi-

epoch ≈ +2.6, offline warmup ≈ +3.4 average points. Finance without any feedback signal: both ACE and DC can *degrade* (FiNER online ACE no-GT 67.3 < base 70.7).

Code / data / license. Code: https://github.com/ace-agent/ace (Apache-2.0; `ace/`, `playbook_utils.py`, `eval/finance/run.py`, `ace-appworld` submodule, `EXTENDING_ACE.md`). Paper: ICLR 2026, 32 pages.

Stated limitations (Appendix B). Relies on a strong Reflector — if it cannot extract meaningful insights the context becomes noisy or harmful; domains where no model can extract insight yield no useful context; like DC, adaptation quality depends on the model's curation ability; not every task needs rich context (HotPotQA prefers concise instructions; Game of 24 needs one rule). Discussion (Sec. 5): longer context does not imply proportionally higher serving cost thanks to KV-cache reuse / compression / offloading; human-readable contexts allow selective unlearning.

Use in our design. The Notebook *is* an ACE playbook: sectioned bullets with ids and helpful/harmful counters, Generator = worker 9B, Reflector + Curator = senior model, merge = deterministic code. Our execution feedback (tests, type checks, verifier cascade) is the "GT" signal ACE shows is necessary to avoid degradation. Change for a single-user local system: refinement should be *lazy* (triggered at a token budget far below 80k, e.g. the core-block budget) and run in idle time; add an explicit UPDATE/RETIRE operation (ACE prompts only ADD) driven by counters; multi-epoch = re-run the Reflector over the week's verified trajectories during consolidation.

PDF: https://arxiv.org/pdf/2510.04618 — Code: https://github.com/ace-agent/ace

1.2 Reflexion: Language Agents with Verbal Reinforcement Learning (Shinn et al., 2023) — arXiv:2303.11366

Problem and claim. LLM agents acting in environments (games, compilers, APIs) cannot cheaply learn by trial and error because RL needs many samples and fine-tuning. Reflexion instead reinforces agents with *linguistic* feedback: after a failed trial the agent writes a verbal self-reflection, stores it in an episodic memory buffer, and conditions the next trial on it. The authors call this a "semantic gradient" (analogy, not a formal object). No weights change.

Headline results. HumanEval (Python) pass@1 **91.0%** vs GPT-4 **80.1%** (prev. SOTA CodeT+GPT-3.5 65.8). HumanEval (Rust, 50 hardest via MultiPL-E) 68.0 vs 60.0. MBPP (Python) **77.1 vs 80.1** (the one loss). MBPP (Rust) 75.4 vs 70.9. LeetcodeHardGym (Python) 15.0 vs 7.5. AlfWorld: ReAct+Reflexion solves **130/134** tasks over 12 trials; ReAct-only plateaus between trials 6–7 at a 22% hallucination rate. HotPotQA (100 questions): CoT(GT) text-davinci-003 0.60→0.77, gpt-3.5-turbo 0.57→0.71, gpt-4 0.68→0.80; ReAct gpt-4 0.39→0.51; adding self-reflection beats adding only the previous trajectory by **8 points**.

Method — equations. No numbered equations. The policy is

$$\pi_\theta\left(a_i \mid s_i\right), \qquad \theta = \{M_a, \mathrm{mem}\}$$

where M_a is the frozen Actor LLM and mem is the long-term memory of self-reflections; "learning" means changing mem only. M_e (Evaluator) maps a trajectory τ_t to a scalar reward r_t; M_{sr} (Self-Reflection) maps $\left(r_t, \tau_t, \mathrm{mem}\right)$ to a text sr_t. Memory is bounded: $|\mathrm{mem}| \leq \Omega$, with Ω "usually 1–3".

Method — algorithm (Algorithm 1, "Reinforcement via self-reflection").

```
Initialize Actor, Evaluator, Self-Reflection: M_a, M_e, M_sr
Initialize policy pi_theta(a_i | s_i), theta = {M_a, mem}
Generate initial trajectory tau_0 using pi_theta
Evaluate tau_0 using M_e  -> r_0
Generate initial self-reflection sr_0 using M_sr
Set mem <- [sr_0]
Set t = 0
while M_e not pass or t < max_trials do
    Generate tau_t = [a_0, o_0, ..., a_i, o_i] using pi_theta
    Evaluate tau_t using M_e
    Generate self-reflection sr_t using M_sr
    Append sr_t to mem        (sliding window of size Omega)
    Increment t
end while
return
```

Short-term memory = the current trial's trajectory; long-term memory = `mem`. Evaluator variants: exact match (reasoning), heuristics (decision-making: same action+observation repeated > 3 cycles, or > 30 actions in the

environment), LLM judge, or **self-generated unit tests** (programming: tests written with CoT plus a natural-language description, each parsed to an AST and discarded if invalid, up to **6** tests per problem). Programming loop: actor emits only a function body; prompt cycles instruction → implementation → unit-test feedback → self-reflection → next instruction; memory limit $\Omega = 1$.

Hyperparameters and setup. Ω: 3 (AlfWorld, HotPotQA), 1 (programming). AlfWorld: 12 trials, 134 tasks, 6 task types, ReAct actor. HotPotQA: 6-shot CoT, 2-shot ReAct, 2-shot self-reflection, temperature 0.7, retry until 3 consecutive failures. Models: GPT-4, gpt-3.5-turbo, text-davinci-003; starchat-beta (Appendix A) showed no gain (0.26 → 0.26).

Ablations that matter. Test-generation quality (Table 2, rates): HumanEval-Py TP 0.99 / FN 0.40 / **FP 0.01** / TN 0.60; MBPP-Py TP 0.84 / FN 0.59 / **FP 0.16** / TN 0.41; HumanEval-Rs FP 0.13; MBPP-Rs FP 0.16. The MBPP loss is attributed to the 16% false-positive rate (bad tests pass → wrong code submitted early); false negatives are less harmful because reflection can repair bad tests. Table 3 (HumanEval-Rust, GPT-4): base 0.60; no test generation 0.52; no self-reflection 0.60; full 0.68 — i.e. reflection without tests gives nothing; tests without reflection hurt.

Code / data / license. https://github.com/noahshinn024/reflexion (now redirects to noahshinn/reflexion). Paper page CC BY 4.0; code license not stated in text. NeurIPS 2023 (not stated on arXiv page).

Stated limitations. Depends on the LLM's self-evaluation ability; can settle in non-optimal local minima (WebShop: cannot overcome minima requiring creative exploration); memory capped by a sliding window; programming agents are bound by their ability to write diverse, comprehensive tests; test-driven approach fails for non-deterministic, impure (API-calling), hardware-dependent or concurrent functions; generated code is executed unvalidated (use isolated environments).

Use in our design. The per-task retry loop (worker attempt → verifier cascade → senior writes a reflection → retry with the reflection in context, $\Omega \leq 3$) is Reflexion. The "lessons become tests" idea follows directly: the Evaluator that wins is a test suite, and the dominant failure is false-positive tests, so every lesson-derived test must be checked to *fail* on the pre-fix code before it is kept. Change: our reflections are not discarded per task; the Curator (ACE) promotes recurring ones into Notebook bullets.

PDF: https://arxiv.org/pdf/2303.11366 — Code: https://github.com/noahshinn024/reflexion

1.3 Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory (Suzgun et al., 2025) — arXiv:2504.07952

Problem and claim. LMs answer each query in isolation and so re-derive insights and repeat mistakes. DC gives a black-box LM a persistent, evolving natural-language memory ("cheatsheet") at inference time, curated by the model itself, with **no ground-truth labels and no human feedback**; strategies and code snippets accumulated on earlier queries transfer to later ones.

Headline results (Table 1, accuracy %; Claude 3.5 Sonnet / GPT-4o; BL = baseline, DC-∅ = DC prompt with empty memory, DR = dynamic retrieval of raw past I/O, DC-Cu = cumulative, DC-RS = retrieval & synthesis):

Benchmark	Claude BL	DC-∅	DR	DC-Cu	DC-RS	4o BL	DC-∅	DR	DC-Cu	DC-RS
AIME 2024	23.3	36.7	43.3	**50.0**	46.7	20.0	36.7	26.7	36.7	**40.0**
AIME 2025	6.7	23.3	23.3	**36.7**	30.0	6.7	10.0	10.0	16.7	**20.0**
AIME 2020–24	6.7	30.1	39.1	38.4	**40.6**	9.8	24.1	24.1	20.3	24.8
Game of 24	12.0	10.0	11.0	14.0	14.0	10.0	19.0	6.0	93.0	**99.0**
GPQA-Diamond	59.6	60.1	63.6	61.1	**68.7**	57.1	57.1	55.1	58.1	57.1
Math Equation Balancer	44.8	56.4	60.4	**100**	97.8	50.0	88.0	100	100	99.2
MMLU-Pro Engineering	61.2	57.2	65.2	66.8	**67.6**	53.2	51.6	48.8	44.0	51.2
MMLU-Pro Physics	74.0	75.6	80.4	77.6	**82.0**	75.6	70.8	75.6	70.4	75.2

Full-History appending (Table 2) is worse than curation: Claude AIME 2024 BL 23.3 / FH 26.7 / DC-Cu 50.0; GPT-4o BL 20.0 / **FH 13.3** / DC-RS 40.0. Smaller models gain much less (Table 3: Claude 3.5 Haiku AIME 2024 10.0 → 36.7 DC-Cu; GPT-4o-mini 16.7 → 20.0 DC-∅). Average output tokens on AIME 2024 (Claude): 370 BL, 494 DC-∅, 1,035 DC-RS, 1,831 DC-Cu.

Method — equations (Sec. 2). Inputs $x_1, ..., x_N$ arrive sequentially; M_i is memory before query i.

$$\tilde{y}_i = \mathrm{Gen}\,(\,x_i, M_i\,) \qquad (\,1\,)$$

$$M_{i+1} = \mathrm{Cur}\left(\,M_i, x_i, \tilde{y}_i\,\right) \qquad (\,2\,)$$

Gen is the generator (answer conditioned on query and memory), Cur the curator (judges whether $\tilde{y}_i$ is correct/generalizable, refines or removes entries, keeps memory compact — with no ground truth). DC-RS updates memory *before* answering:

$$R_i = \mathrm{Retr}(x_i, \{\left(x_j, \tilde{y}_j\right)\}_{j<i}, k) \qquad (\,3\,)$$

$$M_i = \mathrm{Cur}\,(\,M_{i-1}, x_i, R_i\,) \qquad (\,4\,)$$

$$\tilde{y}_i = \mathrm{Gen}\,(\,x_i, M_i\,) \qquad (\,5\,)$$

R_i = top-k past input/output pairs by cosine similarity of OpenAI `text-embedding-3-small` embeddings, $k = 3$.

Method — algorithm. 1. DC-Cu: for each x_i: generate with Eq. 1; curator rewrites the **entire** cheatsheet with Eq. 2 (output is a complete `<cheatsheet>` block with a version number; the prompt warns that anything not copied forward is lost). 2. DC-RS: retrieve $k = 3$ similar past pairs (Eq. 3), curate (Eq. 4), then generate (Eq. 5). 3. Generator prompt: cheatsheet in a `CHEATSHEET:` section before the question; instructed to "Search for and identify any applicable patterns, strategies, or examples within the cheatsheet", write Python for computational steps, answer in `<answer>` tags. 4. Curator prompt: keep high-value strategies, code blocks, insights, reusable patterns and still-effective older techniques; discard redundant, trivial or over-specific items; replace outdated entries; each entry is a `<memory_item>` with `<description>`, `<example>` and a **usage count**; sections: reusable code and solution strategies, general heuristics, optimization and edge cases, specialized theorems; target length "circa 2000–2500 words".

Hyperparameters and setup. $k = 3$ ($k = 5, 7$ insignificant gains); embedding `text-embedding-3-small`; temperature and max tokens not in fetched text. Models: Claude 3.5 Sonnet, GPT-4o, Claude 3.5 Haiku, GPT-4o-mini; R1-style models gave verbose output and minimal gains.

Ablations that matter. DC-Cu > DC-RS on AIME (Claude); DC-RS > DC-Cu on GPQA-Diamond and MMLU-Pro; FH hurts GPT-4o; majority voting over BL does not close the gap (Table 4: MV(BL) 23.33 vs DC-Cu 50.0 on AIME 2024); curriculum ordering (related/simpler problems first) helps qualitatively; cross-model memory transfer gives "mixed results"; models sometimes abbreviate prior memory ("Previous content [...] preserved") instead of rewriting it — the seed of context collapse that ACE measured.

Code / data / license. https://github.com/suzgunmirac/dynamic-cheatsheet (code license not stated in text). Preprint.

Stated limitations. Small models gain little because they rarely produce correct solutions to curate; retrieval can add noise (GPT-4o on GPQA); models struggle to regenerate long memory; strictly sequential — poor fit for parallel batch jobs.

Use in our design. Take the `<memory_item>{description, example, usage_count}` content shape and the curator's keep/discard rules; take the finding that a *curated* memory beats raw full history (FH) and raw retrieval (DR). Reject the full-rewrite update (Eq. 2) because of collapse; and because our worker is a 9B, curation must be done by the senior model (DC shows weak models curate badly). Our verifier gives us the ground truth DC lacks, so we should feed it to the curator.

PDF: https://arxiv.org/pdf/2504.07952 — Code: https://github.com/suzgunmirac/dynamic-cheatsheet

1.4 A-MEM: Agentic Memory for LLM Agents (Xu et al., 2025) — arXiv:2502.12110

Problem and claim. Existing agent memory systems store and retrieve but do not *organize*; fixed schemas limit cross-task use. A-MEM, inspired by Zettelkasten, builds an interconnected note network: each new memory becomes a structured note with LLM-generated keywords, tags and context; it is linked to similar notes; and adding it can **evolve** the context/tags of its neighbours. Works on six foundation models including 1B–3B local ones.

Headline results (LoCoMo, F1 / BLEU-1 %, Table 1). GPT-4o-mini: A-Mem multi-hop **27.02/20.09** (MemGPT 26.65/17.72, LoCoMo 25.02/19.75), temporal **45.85/36.67** (MemGPT 25.52/19.44), open-domain 12.14/12.00, single-hop 44.65/37.06 (MemGPT 41.04/34.34), adversarial 50.03/49.47 (LoCoMo 69.23/68.75 higher), **2,520 tokens/query** vs ~16,900 for LoCoMo/MemGPT. GPT-4o: A-Mem 32.86/23.76 multi-hop, 39.41/31.23 temporal, 17.10/15.84 open, 48.43/42.97 single-hop (LoCoMo 61.56/54.19 higher), 36.35/35.53 adversarial, **1,216 tokens**, mean rank 1.6 vs LoCoMo 2.0, MemGPT 2.4. Small models: Qwen2.5-1.5B LoCoMo→A-Mem multi-hop 9.05→18.23, temporal 4.25→24.32, single-hop 11.15→23.63, 1,300 tokens; Llama 3.2-1B multi-hop 11.25→19.06, single-hop 12.86→28.51, 1,376 tokens. DialSim: A-Mem F1 3.45 / BLEU-1 3.37 vs LoCoMo 2.55/3.13, MemGPT 1.18/1.07. Cost: ≈1,200 tokens per memory operation (< $0.0003 with commercial APIs); latency ≈5.4 s (GPT-4o-mini), ≈1.1 s (Llama 3.2-1B, one GPU).

Method — equations (Sec. 3). Note:

$$m_i = \{c_i, t_i, K_i, G_i, X_i, e_i, L_i\} \quad (1)$$

c_i original content, t_i timestamp, K_i keywords, G_i tags, X_i contextual description (all LLM-generated), e_i dense embedding, L_i linked memories.

$$K_i, G_i, X_i \leftarrow \mathrm{LLM}\left(c_i \parallel t_i \parallel P_{s1} \right) \quad (2)$$

$$e_i = f_{\mathrm{enc}}\left[\mathrm{concat}\left(c_i, K_i, G_i, X_i \right) \right] \quad (3)$$

P_{s1} note-construction prompt; f_{enc} text encoder (all-MiniLM-L6-v2). Link generation for new note m_n:

$$s_{n,j} = \frac{e_n \cdot e_j}{\| e_n \| \ \| e_j \|} \quad (4)$$

$$\mathcal{M}^n_{\mathrm{near}} = \{ m_j \mid \mathrm{rank}\left(s_{n,j} \right) \leq k,\ m_j \in \mathcal{M} \} \quad (5)$$

$$L_i \leftarrow \mathrm{LLM}\left(m_n \parallel \mathcal{M}^n_{\mathrm{near}} \parallel P_{s2} \right) \quad (6)$$

Memory evolution of each neighbour m_j:

$$m_j^* \leftarrow \mathrm{LLM}\left(m_n \parallel \mathcal{M}^n_{\mathrm{near}} \backslash m_j \parallel m_j \parallel P_{s3} \right) \quad (7)$$

Retrieval for query q:

$$e_q = f_{\mathrm{enc}}\left(q \right) \quad (8)$$

$$s_{q,i} = \frac{e_q \cdot e_i}{\| e_q \| \ \| e_i \|} \quad (9)$$

$$\mathcal{M}_{\mathrm{retrieved}} = \{ m_i \mid \mathrm{rank}\left(s_{q,i} \right) \leq k,\ m_i \in \mathcal{M} \} \quad (10)$$

Method — algorithm (insert path; prompts from Appendix B). 1. **Note construction** (P_{s1}): LLM returns JSON `{keywords: [...], context: "...", tags: [...]}` — keywords ordered most→least important, exclude speaker names/times, ≥3, no redundancy; context = one sentence (main topic, key points, audience/purpose); tags = broad themes (domain, format, type), ≥3. 2. Embed with Eq. 3; find k nearest existing notes (Eqs. 4–5). 3. **Link generation** (P_{s2}): LLM, framed as "memory evolution agent", sees the new note's context/content/keywords plus neighbours and decides links. 4. **Memory evolution** (P_{s3}): LLM returns JSON `{should_evolve: bool, actions: [...], suggested_connections: [neighbour ids], tags_to_update: [...], new_context_neighborhood: [one per memory], new_tags_neighborhood: [one list per memory]}`, ordered [new memory, neighbour 1..n]. Actions in prose: `strengthen` (connect new note to a neighbour, update its tags) and `update_neighbor` (revise neighbours' context and tags); the JSON example also lists `merge` and `prune`. Replace each m_j by m_j^*. 5. **Retrieval**: Eqs. 8–10, top-k by cosine; retrieved notes are placed in the agent's prompt.

Hyperparameters and setup. Embedding all-MiniLM-L6-v2 everywhere; retrieval $k = 10$ default; per-category tuned k (Table 8, GPT-4o-mini and GPT-4o): multi-hop 40, temporal 40, open-domain 50, single-hop 50, adversarial 40; k sweep 10–50 plateaus; same k reused for neighbour candidates; local models via Ollama + LiteLLM structured output; GPT via structured-output API; temperature not stated.

Ablations that matter (GPT-4o-mini, F1/BLEU-1): w/o link generation and memory evolution 9.65/7.09 multi-hop, 13.28/10.30 single-hop; w/o memory evolution 21.35/15.13 multi-hop, 39.17/34.70 single-hop; full 27.02/20.09 and 44.65/37.06. Linking carries most of the gain; evolution adds ~6 F1 on multi-hop and ~14 on temporal. Retrieval time grows minimally with memory size (Sec. 4.6).

Code / data / license. Eval: https://github.com/WujiangXu/A-mem; system: https://github.com/WujiangXu/A-mem-sys (licenses not in fetched text). NeurIPS 2025.

Stated limitations. Organization quality is bounded by the underlying LLM; text-only.

Use in our design. Every Notebook entry gets A-MEM's `{keywords, tags, context, embedding, links}` metadata, and insertion runs the two-step link + evolve procedure with k bounded (10). This is how a new lesson updates the wording/tags of old related ones without a full rewrite — ACE's missing UPDATE operation. The 1B–3B results show the embedding+link machinery pays off even for small models, but the LLM steps (keywords, evolution) should run on the senior model in idle time, not on the worker in the hot path.

PDF: https://arxiv.org/pdf/2502.12110 — Code: https://github.com/WujiangXu/A-mem-sys

1.5 Agent Workflow Memory (Wang et al., 2024) — arXiv:2409.07429

Problem and claim. LM web agents fail at long-horizon tasks because they learn from a fixed example set and never from their own successes. AWM induces reusable **workflows** (goal description + abstracted step sequence) from past experiences and puts them in the agent's memory, offline (from training examples) or online (from its own test-time trajectories judged successful by a model evaluator).

Headline results. Relative gains **+24.6%** Mind2Web step success rate, **+51.1%** WebArena success rate. WebArena (GPT-4, gpt-4-0613, temp 0.0): BrowserGym (ax-tree) 15.0 → **AWM 35.5** total SR (SteP with human-written workflows 33.0; published BrowserGym 23.5); steps per task 7.9 → **5.9**. Per site (BrowserGym→AWM): Shopping 17.2→30.8, CMS 14.8→29.1, Reddit 20.2→50.9, GitLab 19.0→31.8, Maps 25.5→43.3. Mind2Web (GPT-4; step SR / element acc; MindAct → AWM offline → AWM online): cross-task 36.2/41.6 → 45.1/50.6 → 43.6/50.0; cross-website 30.1/35.8 → 33.7/41.4 → 33.9/42.1; cross-domain 18.6/21.6 → 32.6/36.4 → **35.5/40.9**. Online AWM's advantage grows with train–test distribution gap.

Method — equations (Sec. 2, notation as written). Experience $e = (q, P^e)$, $P^e = \left(p_1^e, ..., p_n^e\right)$, step $p = (o, a)$ (observation, action). Workflow $w = \left(d_j, P_j^d\right)$: description plus step sequence; in text each workflow step has a state description, the agent's reasoning and an executable action. Induction

$$I(\mathcal{E}) \rightarrow \mathcal{W} = \{w\} = \{\left(d_j, P_j^d\right)\}, \qquad \mathcal{E} = \{e_i\}_{i=1}^m$$

Memory update: offline $M + \mathcal{W} \rightarrow M_w$; online $M^t + \{w^t\} \rightarrow M^{t+1}$.

Method — algorithm (prose; no algorithm box). 1. **Induction** I: an LM is prompted with one or more experiences to extract common sub-routines, steered toward fine-grained reusable sub-tasks, and told to abstract example-specific values into placeholders (e.g. "dry cat food" → `{product-name}`). Output split on double line breaks; each chunk stored as one workflow. 2. **Offline**: concatenate all training examples of a website into one prompt, induce once; test-time memory = base memory + workflows. 3. **Online**: for each incoming query, run the agent; a model-based evaluator (Pan et al., 2024) labels the trajectory success/failure; if success, induce workflows from it and add to memory before the next query. 4. Workflows are added as auxiliary context (system prompt / memory), not as replacements for the few-shot demonstrations.

Hyperparameters and setup. WebArena with BrowserGym (accessibility tree), GPT-4-0613, temperature 0.0; Mind2Web with MindAct, gpt-3.5-turbo and GPT-4, temperature 0.0; few-shot counts not in fetched text.

Ablations that matter. Rule-based vs LM-based induction: WebArena 35.6 vs 35.5 SR, 6.3 vs 5.9 steps; Mind2Web cross-task step SR 43.4 vs 45.1, task SR 2.0 vs 4.8. Workflows as text vs code: step SR 45.4 vs 45.1. Abstracted workflows vs concrete examples (Synapse): +5.0 element accuracy, +4.0 step SR. Online: cumulative success rises fast over the first ~40 WebArena examples then stabilizes. Workflow actions were used in only **18.5%** of tasks; following workflows can cause divergence errors and slightly lower action F1; induction from imperfect predicted trajectories can produce wrong workflows that hurt, especially in-domain.

Code / data / license. https://github.com/zorazrw/agent-workflow-memory (license not in fetched text). ICML 2025 (PMLR v267).

Stated limitations. Low workflow utilization; divergence when following a workflow; fixed workflows break when the environment changes dynamically (pop-up menus).

Use in our design. Playbook entries of kind `workflow` = `{description, steps[{state, reasoning, action}]}` with placeholders, induced *only* from trajectories the verifier cascade passed (AWM's noisy-evaluator warning becomes a hard rule: no induction from unverified runs). Rule-based induction being as good as LM-based means a cheap deterministic inducer (collapse repeated tool-call patterns) can run on the worker; the senior only abstracts values into placeholders. Workflows are the bridge to Section B's Skill tier: a workflow that stays helpful gets promoted to a tested skill.

PDF: https://arxiv.org/pdf/2409.07429 — Code: https://github.com/zorazrw/agent-workflow-memory

1.6 Generative Agents: Interactive Simulacra of Human Behavior (Park et al., 2023) — arXiv:2304.03442

Problem and claim. Believable long-running agents need to remember a growing stream of experience, synthesize it into higher-level reflections and retrieve the right pieces to plan. The architecture: a **memory stream** of natural-language records, a **retrieval** function scoring recency, importance and relevance, **reflection** that periodically abstracts observations into insights with evidence pointers, and **planning** with recursive decomposition. Evaluated with 25 agents in a Sims-like sandbox.

Headline results. Controlled evaluation (100 participants, within-subjects, TrueSkill ratings μ / σ): full architecture **29.89 / 0.72**; no reflection 26.88 / 0.69; no reflection + no planning 25.64 / 0.68; human crowdworker 22.95 / 0.69; no observation/reflection/planning 21.21 / 0.70. Kruskal-Wallis $H(4) = 150.29$, $p < 0.001$; Dunn post-hoc (Holm-Bonferroni) all pairs $p < 0.001$ except crowdworker vs fully ablated; full vs no-memory Cohen's $d = 8.16$. End-to-end over two game days: awareness of Sam's candidacy 1/25 (4%) → 8/25 (32%); Isabella's party 1/25 → 13/25 (52%); relationship network density 0.167 → 0.74; 12 agents invited, 5 attended; 6/453 (1.3%) hallucinated awareness responses.

Method — equations (Sec. 4.1). Each memory object = natural-language description + creation timestamp + most-recent-access timestamp. For a query memory, each candidate memory gets

$$\text{recency} = 0.995^{\,h}$$

where h is the number of sandbox game hours since the memory was last retrieved ("Our decay factor is 0.995");

$$\text{importance} \in \{1, \ldots, 10\}$$

rated by the LLM at creation with the prompt "On the scale of 1 to 10, where 1 is purely mundane (e.g., brushing teeth, making bed) and 10 is extremely poignant (e.g., a break up, college acceptance), rate the likely poignancy of the following piece of memory." (examples: cleaning the room → 2, asking a crush on a date → 8);

$$\text{relevance} = \cos\left(\mathbf{v}_{\text{memory}}, \mathbf{v}_{\text{query}}\right)$$

cosine similarity of LLM embeddings of the description and the query. Each of the three is min-max scaled to $[0, 1]$ over the candidates, then

$$\text{score} = \alpha_{\text{recency}} \cdot \text{recency} + \alpha_{\text{importance}} \cdot \text{importance} + \alpha_{\text{relevance}} \cdot \text{relevance}, \qquad \alpha_{\text{recency}} = \alpha_{\text{importance}} = \alpha_{\text{relevance}} = 1.$$

Top-ranked memories that fit the context window are included.

Method — algorithm. *Reflection* (Sec. 4.2): trigger when the **sum of importance scores of the latest perceived events exceeds 150** (≈2–3 reflections per day). Then: (1) take the **100 most recent** memory records and ask for the "3 most salient high-level questions we can answer about the subjects in the statements"; (2) use each question as a retrieval query; (3) prompt "What 5 high-level insights can you infer from the above statements? (example format: insight (because of 1, 5, 3))"; (4) store each insight as a reflection with pointers to the cited records. Reflections can cite reflections → a tree whose leaves are observations. *Planning* (Sec. 4.3): generate a day agenda of 5–8 chunks from the agent summary and previous-day summary; recursively decompose into hour-long then 5–15-minute actions; plans live in the memory stream and are retrieved like observations; at each step the LM decides to continue or react, and on reaction the plan is regenerated from that time onward. The agent's "summary description" is a dynamically generated paragraph that evolves with experience.

Hyperparameters and setup. gpt-3.5-turbo; 25 agents; decay 0.995 per game hour; reflection threshold 150; 100 recent records; 3 questions; 5 insights; importance 1–10; interview categories self-knowledge, memory, plans, reactions, reflections (5 questions each).

Ablations that matter. Removing reflection costs ≈3 TrueSkill points; removing reflection+planning ≈4.3; removing all memory ≈8.7 and falls below humans. Qualitative failures: retrieval misses or fragments (Tom remembers the election talk but not the party), embellishment (rare fabrication, e.g. "he's going to make an announcement tomorrow"), odd location choices as memory grows, norm misunderstandings, overly formal/cooperative style from instruction tuning. Robustness concerns: prompt hacking, memory hacking, inherited bias.

Code / data / license. https://github.com/joonspk-research/generative_agents (code license not in fetched text; arXiv non-exclusive license). UIST 2023 (not stated on arXiv page).

Stated limitations. Cost and time of the simulation (not quantified in fetched text); the above failure modes; parasocial and misuse risks.

Use in our design. The Notebook's retrieval score is this formula with the three weights exposed as config and "game hours" replaced by hours of assistant activity (or task count) since last access, so dormant entries decay. Importance (1–10, rated by the senior) is also the **promotion** signal: cumulative importance of new notes since the last consolidation is the trigger for an idle-time reflection pass (threshold 150 is a starting value to recalibrate for our importance distribution). Reflections with evidence pointers are exactly "lessons citing facts", which lets a lesson be invalidated when its evidence is retired.

PDF: https://arxiv.org/pdf/2304.03442 — Code: https://github.com/joonspk-research/generative_agents

1.7 MemGPT: Towards LLMs as Operating Systems (Packer et al., 2023) — arXiv:2310.08560

Problem and claim. Fixed context windows break long conversations and long-document analysis. MemGPT proposes **virtual context management**: like OS paging, the LLM itself moves information between a small *main context* and large *external context* via function calls, giving the illusion of unbounded context. Control flow is event-driven with heartbeats for multi-step retrieval.

Headline results. Deep Memory Retrieval (multi-session chat, Table 2; accuracy / ROUGE-L recall): GPT-3.5 Turbo 38.7% / 0.394 → **MemGPT 66.9% / 0.629**; GPT-4 32.1% / 0.296 → **92.5% / 0.814**; GPT-4 Turbo 35.3% / 0.359 → **93.4% / 0.827**. Conversation opener (Table 3; SIM-1 / SIM-3 / SIM-H): human 0.800 / 0.800 / 1.000; MemGPT GPT-3.5 0.830 / 0.812 / 0.817; GPT-4 0.868 / 0.843 / 0.773; GPT-4 Turbo 0.857 / 0.828 / 0.767. Document QA (NaturalQuestions-Open, 50 questions, 2018 Wikipedia): MemGPT(GPT-4) accuracy unaffected by the number of retrieved documents (0–200) while fixed-context GPT-4 is capped by its window; exact values only in Fig. 5 (≈0.1–0.7 range). Nested key-value retrieval (140 UUID pairs ≈ 8k tokens, nesting 0–4, 30 orderings): GPT-3.5 → 0% at 1 level, GPT-4/4-Turbo → 0% by 3 levels, MemGPT(GPT-4) essentially flat across 0–4; MemGPT with GPT-4 Turbo and GPT-3.5 degrade from 2 levels.

Method — equations. None. Quantities are thresholds on prompt-token count: warning at ≈ **70%** of the window, flush at **100%**, evict ≈ **50%** of the window (all given as "e.g." examples).

Method — algorithm (Sec. 2). 1. **Main context** (what the LLM sees), three contiguous parts: read-only *system instructions* (control flow, memory tiers, function usage); *working context* — "a fixed-size read/write block of unstructured text, writeable only via MemGPT function calls" (key facts, user preferences); *FIFO queue* — rolling message history whose **first entry is a recursive summary of evicted messages**. 2. **External context**: *recall storage* (full message database; evicted messages stay there indefinitely, searchable) and *archival storage* (arbitrary-length read/write text objects; in experiments PostgreSQL + pgvector, HNSW index, `text-embedding-ada-002` cosine similarity). 3. **Queue manager**: when prompt tokens exceed the warning count (e.g. 70%) insert a memory-pressure system message so the model saves important facts to working/archival memory; when they exceed the flush count (100%) evict a fixed number of messages (e.g. 50% of the window), generate a new recursive summary from (old summary + evicted messages), keep evicted messages in recall storage. 4. **Function set** (as shown in figures, dotted form): `working_context.append(text)`, `working_context.replace(old, new)`, `recall_storage.search(query)`, `archival_storage.search(query, page=n)`; search results are paginated so they cannot overflow the window and the model can page through them. 5. **Control flow**: events (user message, system warning, timed trigger, function result) cause inference; a function call may carry `request_heartbeat=true` to request immediate re-inference (function chaining); otherwise the processor yields. Errors (e.g. appending to a full working context) are fed back as messages. Memory management is "entirely self-directed".

Hyperparameters and setup. gpt-4-0613 (8,192-token window) as the main model; GPT-3.5 Turbo and GPT-4 Turbo also tested; MSC dataset extended with a sixth session containing one QA pair; LLM judge for consistency + ROUGE-L recall; working-context size and page size not stated.

Ablations that matter. No formal ablation; the model comparison is the ablation: MemGPT with GPT-3.5 is "significantly degraded" on document QA, attributed to weak function calling; MemGPT sometimes stops paging before exhausting results.

Code / data / license. https://research.memgpt.ai (points to the MemGPT/Letta repository; license not stated in text). Preprint (v2, Feb 2024).

Stated limitations. Needs a model that follows function-call protocols reliably; degraded with weaker models; may terminate retrieval early.

Use in our design. The **core block** is MemGPT's working context: a fixed-size always-in-context text edited only through `append`/`replace` operations issued by the Curator (not free-form by the worker). Session transcript = FIFO queue with a recursive summary head; Notebook + repo index = archival; raw logs = recall. Adopt the

70%/100%/50% thresholds as initial values scaled to the worker's window. Change: since a 9B worker is in the GPT-3.5 regime for tool use, memory-edit functions should be issued by the senior or by deterministic code, and search pages should be small and explicitly enumerated so the worker cannot stop early unnoticed.

PDF: https://arxiv.org/pdf/2310.08560 — Code: https://research.memgpt.ai

1.8 Sleep-time Compute: Beyond Inference Scaling at Test-time (Lin et al., 2025) — arXiv:2504.13171

Problem and claim. Test-time scaling assumes the context arrives with the query and all reasoning happens at query time (latency and cost). In many applications (agents, document QA, coding) the context exists *before* the query. **Sleep-time compute** reasons about the context offline — drawing inferences, pre-computing results, anticipating questions — and stores a re-represented context so test-time work is smaller; the cost amortizes over multiple queries on the same context.

Headline results. Stateful GSM-Symbolic (P1 5,000 / P2 2,500 examples) and Stateful AIME (60 problems, 2024+2025): same accuracy with **≈5× fewer test-time tokens**; scaling sleep-time compute raises accuracy by up to **+13%** (GSM-Symbolic, largest on P2 with GPT-4o) and **+18%** (AIME) at similar test-time budgets. Multi-Query GSM-Symbolic (P1: 1,095 contexts / 12,043 questions; P2: 500 / 5,497; ≈11 questions per context, o3-mini-generated): average cost per query falls up to **2.5×** at 10 queries per context. Sleep-time beats pass@k at equal test-time tokens (pass@k assumes an oracle verifier). SWE-Features (PRs touching ≥3 .py/.js files; metric F1 of modified-file set): at low test-time budgets sleep-time gives higher F1 with up to ≈1.5× fewer test-time tokens; at high budgets standard test-time scaling wins (higher precision; sleep-time agents edit more files).

Method — equations (Sec. 2). Standard test-time compute:

$$T_B\left(q, c\right) \rightarrow a$$

T = a test-time method (extended CoT, best-of-N …) with budget B, q query, c context, a answer. Sleep-time:

$$S\left(c\right) \rightarrow c', \qquad T_b\left(q, c'\right) \rightarrow a, \qquad b \ll B$$

S = the offline process (with its own budget) producing the re-represented context c'. Multi-query amortization: N queries share c, so the cost of c' is divided by N. Cost model (linear): total cost $= \text{tokens}_{\text{sleep}} + t \cdot \text{tokens}_{\text{test}}$ with $t = 10$ (latency-optimized inference ≈10× dearer than throughput-optimized, citing a Databricks benchmark); no closed form beyond this.

Method — algorithm. 1. **Sleep phase** (general prompt, Fig. 17): the model repeatedly calls a `rethink_memory` function to reorganize and consolidate memory, drawing logical conclusions and hypotheses from the context and writing them into a *rethink memory block*. AIME variant (Fig. 18): "you see only part of a problem; make as many inferences as possible, predict how the rest might go, compute results that could help later." Scale by k parallel generations of c' (non-reasoning models; $k = 5$ generally beats $k = 10$) or by reasoning effort (o1, o3-mini). 2. **Test phase**: Letta persona prompt; the model must check the rethink memory block before answering and avoid recomputing what is there; answer via `send_message`. 3. Test-time scaling knobs: GPT-4o/4o-mini five verbosity levels at temperature 0 (level 0 "The answer is …" one sentence; level 1 one explanatory sentence; levels 2–3 only necessary tokens; level 4 step-by-step and recompute remembered numbers); o-series `reasoning_effort`; Claude 3.7 extended-thinking budget; DeepSeek-R1 budget forcing (Muennighoff et al., 2025).

Hyperparameters and setup. Models: GPT-4o-mini, GPT-4o (GSM); o1, o3-mini, Claude 3.7 Sonnet Extended Thinking, DeepSeek-R1 (AIME); predictability scored with Llama-2-70B base log-probability of the question given the context, binned into five quantiles; runs averaged over 3 (o1, o3-mini, R1) or 10 (Claude).

Ablations that matter. Accuracy gap (sleep-time minus baseline) widens monotonically with query predictability (five quantiles, P1 and P2); at high test-time budgets the baseline slightly overtakes on GSM; o1 shows limited gains on AIME; $k = 5$ vs $k = 10$ sleep generations: 5 better.

Code / data / license. https://github.com/letta-ai/sleep-time-compute (license not in fetched text). Preprint (Letta / UC Berkeley).

Stated limitations. Works only when queries are predictable from the context; how to split budget between sleep and test time is open; assumes a two-phase interaction (no multi-round edits, no variable idle periods); outputs as synthetic training data untested.

Use in our design. Idle-time consolidation is literally $S\left(c\right)$: when the user is away, the senior model (in RAM, otherwise idle) runs `rethink_memory` over the Notebook, the repo index and the week's trajectories — merging

duplicates, decaying stale entries, promoting recurring reflections to playbook bullets, and pre-computing repo facts (module maps, invariants, likely next tasks). A single developer's repo makes queries highly predictable, which is the regime where the paper shows the largest gains. Change: t is not a dollar ratio but interactive latency vs free idle compute, so run sleep-time generously; keep the SWE-Features caveat that sleep-time agents over-edit — pre-computed suggestions must stay advisory and verified.

PDF: https://arxiv.org/pdf/2504.13171 — Code: https://github.com/letta-ai/sleep-time-compute

2. Design extraction for the module

2.1 Algorithms to implement (consolidated pseudocode)

Algorithm N1 — Per-task loop (Reflexion retry inside an ACE delta cycle)

```
function runTask(task, notebook, core):
    retrieved = retrieve(notebook, query = task.description, k = K_RETRIEVE)   # Alg. N2
    mem = []                                                                   # Reflexion long-term mem,
|mem| <= OMEGA
    for t in 0 .. MAX_TRIALS-1:
        traj = Worker9B.generate(core, retrieved, mem, task)                   # Generator; must return
used_bullet_ids
        verdict = VerifierCascade.evaluate(traj)                               # Evaluator: tests, types,
lint, senior review
        if verdict.pass: break
        sr = Senior.reflect(traj, verdict, retrieved)                          # Reflector: JSON
{error_identification, root_cause_analysis, correct_approach, key_insight, bullet_tags[{id, tag in
helpful|harmful|neutral}]}
        mem = (mem + [sr])[-OMEGA:]
    delta = Senior.curate(traj, verdict, mem, retrieved)                       # Curator: JSON {operations:
[{type: ADD|UPDATE|RETIRE|LINK, section, content, ...}]}  -- model never assigns ids
    mergeDelta(notebook, delta, sr.bullet_tags)                                # Alg. N3, deterministic
    if verdict.pass and isMultiStep(traj): induceWorkflows(traj)               # Alg. N5 (AWM: only from
verified-successful trajectories)
    importanceSinceConsolidation += sum(importance(op) for op in delta.ADD)   # Generative Agents trigger
```

Algorithm N2 — Retrieval (Generative Agents score over A-MEM notes)

```
function retrieve(notebook, query, k):
    e_q = embed(query)                                                         # same encoder as notes (A-MEM
Eq. 8)
    cands = notebook.entries.filter(status != retired)
    for m in cands:
        m.rel  = cosine(e_q, m.embedding)                                      # A-MEM Eq. 9 / GA relevance
        m.rec  = DECAY ** hoursSinceLastAccess(m)                              # GA recency, DECAY = 0.995
        m.imp  = m.importance                                                  # GA importance, 1..10, rated
at creation by Senior
    minmax-normalize rel, rec, imp over cands to [0,1]
    score(m) = A_REC*rec + A_IMP*imp + A_REL*rel                               # GA, all alphas = 1 initially
    top = topK(cands, score, k); expand each with its linked notes L_i up to LINK_HOPS=1
    for m in top: m.lastAccess = now                                           # recency is "since last
retrieved"
    return top (truncated to the context budget, highest score first)
```

Algorithm N3 — Deterministic delta merge + grow-and-refine (ACE + A-MEM evolution)

```
function mergeDelta(notebook, delta, bullet_tags):
    for tag in bullet_tags:                                                  # ACE counters
        if tag.tag == helpful: notebook[tag.id].helpful += 1
        if tag.tag == harmful: notebook[tag.id].harmful += 1
    for op in delta.operations:
        switch op.type:
          ADD:
            id = nextId(op.section)                                          # "<section-prefix>-<5
digits>"
            note = buildNote(id, op.content)                                 # A-MEM Eq. 1–3: Senior fills
keywords, tags, context; embed(concat(content, keywords, tags, context))
            near = topK(notebook, cosine(note.embedding, .), K_NEAR)         # A-MEM Eq. 4–5
            dup  = near.filter(sim >= DEDUP_THRESHOLD)                       # ACE analyzer default 0.9
            if dup: notebook[dup[0]].helpful += 1; notebook[dup[0]].aliases += op.content; continue   #
grow, don't duplicate
            note.links = Senior.linkDecision(note, near)                     # A-MEM Eq. 6 (may be deferred
to sleep)
            notebook.add(note); enqueueEvolve(note, near)                    # A-MEM Eq. 7 runs in sleep
(Alg. N4)
          UPDATE:  notebook[op.id].content = op.content; re-embed; bump version; keep history
          RETIRE:  notebook[op.id].status = retired (never delete; keep for provenance)
          LINK:    notebook[op.from].links.add(op.to)
    if tokens(notebook.section(core)) > LAZY_REFINE_BUDGET: scheduleRefine()  # ACE lazy refinement
    appendJournal(delta)                                                     # append-only log; the file is
never rewritten wholesale
```

Algorithm N4 — Idle-time consolidation (Sleep-time compute + Generative Agents reflection + A-MEM evolution)

```
function sleep(notebook, core, repoIndex, weekTrajectories):
    # 1. Evolve neighbours of notes added since last sleep (A-MEM Eq. 7)
    for (note, near) in evolveQueue:
        r = Senior.evolve(note, near)   # JSON {should_evolve,
actions[strengthen|update_neighbor|merge|prune], suggested_connections, tags_to_update,
new_context_neighborhood, new_tags_neighborhood}
        apply r as UPDATE/LINK/RETIRE ops through mergeDelta (deterministic)
    # 2. Reflection (Generative Agents), triggered by importanceSinceConsolidation > REFLECT_THRESHOLD
    recent = notebook.mostRecent(N_RECENT)
    questions = Senior.ask("3 most salient high-level questions", recent)
    for q in questions:
        ev = retrieve(notebook, q, K_RETRIEVE)
        insights = Senior.ask("5 high-level insights ... (because of i, j, k)", ev)
        for ins in insights: mergeDelta(ADD lesson with evidence = cited ids)
    importanceSinceConsolidation = 0
    # 3. Decay / promote / retire (counters from ACE, usage count from DC)
    for m in notebook:
        if m.harmful > m.helpful and m.harmful >= MIN_EVIDENCE: RETIRE m
        if m.kind == lesson and m.helpful >= PROMOTE_HELPFUL and hasTest(m): promote to playbook / Skill-
tier candidate
        if m.kind == reflection and ageDays(m) > REFLECTION_TTL and m.helpful == 0: RETIRE m
    # 4. Multi-epoch re-reflection (ACE): re-run Reflector over the week's verified trajectories, batch size
1, at most EPOCHS passes
    # 5. Sleep-time pre-computation (S(c) -> c'): Senior rethinks repoIndex + core -> rewrites the core
block summary, precomputes module map, invariants, likely next tasks; store as notes of kind `fact` with
source pointers
    # 6. Lessons -> tests: for each lesson with a reproducible failure, Senior writes a test; keep it only
if it FAILS on the pre-fix snapshot and PASSES on the fixed one (Reflexion FP guard)
    # 7. Rebuild core block (Alg. N6) and the per-section token budgets; never rewrite the Notebook file —
only apply ops.
```

Algorithm N5 — Workflow induction (AWM)

```
function induceWorkflows(traj):                                              # only called with
verifier.pass == true
    chunks = ruleInduce(traj)            # collapse repeated (observation, action) patterns into sub-
routines (rule-based ~= LM-based in AWM)
    for ch in chunks:
        w = Senior.abstract(ch)          # {description, steps:[{state, reasoning, action}]}; replace
literals with {placeholders}
        mergeDelta(ADD workflow w, section = "workflows")
```

Algorithm N6 — Core block / queue manager (MemGPT)

```
core = { system_instructions (read-only), working_context (fixed size, edited only by append/replace ops),
fifo_queue (head = recursive summary) }
onEveryTurn:
    if promptTokens > WARN_FRAC * WINDOW:  insert system message "memory pressure: save important facts to
the Notebook"
    if promptTokens > FLUSH_FRAC * WINDOW:
        evicted = fifo.popOldest(until EVICT_FRAC * WINDOW freed)
        fifo.head = Senior.summarize(fifo.head + evicted)                    # recursive summary
        recall.store(evicted)                                                # raw log, searchable
searches over Notebook/recall return fixed-size pages; the caller enumerates pages explicitly.
```

2.2 Equations and quantities the implementation must compute

1. Cosine similarity (A-MEM Eq. 4/9, DC retrieval, GA relevance): $s\left(a,b\right)=\frac{a\cdot b}{\left\|a\right\|\ \left\|b\right\|}$.
2. Recency (GA): $\mathrm{recency}\left(m\right)=\gamma^{h\left(m\right)}$, $\gamma=0.995$, $h\left(m\right)$ = hours (or task-steps) since last retrieval of m.
3. Importance (GA): integer in $\left[1,10\right]$ from the senior at creation; cumulative $\sum\mathrm{importance}$ of new notes since last consolidation compared with threshold $\theta_{\mathrm{reflect}}=150$.
4. Retrieval score (GA): $\mathrm{score}=\alpha_r\widehat{\mathrm{rec}}+\alpha_i\widehat{\mathrm{imp}}+\alpha_v\widehat{\mathrm{rel}}$, hats = min-max normalized over candidates, $\alpha_r=\alpha_i=\alpha_v=1$.
5. Note embedding (A-MEM Eq. 3): $e_i=f_{\mathrm{enc}}\left[\mathrm{concat}\left(c_i,K_i,G_i,X_i\right)\right]$ — embed content *plus* keywords, tags and context, not content alone.
6. Neighbour set (A-MEM Eq. 5): top-k by cosine, $k=10$; duplicate if $s\geq0.9$ (ACE analyzer default).
7. Delta update (ACE / DC Eq. 2 negated): $M_{i+1}=\mathrm{merge}\left(M_i,\Delta_i\right)$ with $\Delta_i=\mathrm{Cur}\left(\mathrm{Refl}\left(\tau_i,r_i\right),M_i\right)$ — never $M_{i+1}=\mathrm{LLM}\left(M_i,\ldots\right)$ (full rewrite).
8. Counters (ACE): per bullet `helpful`, `harmful`; retire when `harmful > helpful` with enough evidence; DC `usage_count` per entry.
9. Reflexion memory bound: $\left|\mathrm{mem}\right|\leq\Omega$, $\Omega\in\{1,2,3\}$ per task.
10. Collapse detector (ACE case study): flag any consolidation step where `tokens(notebook)` drops by more than a configured fraction (the paper's collapse was 18,282 → 122, i.e. −99.3%) and refuse to commit it.
11. Context budget thresholds (MemGPT): warn at $0.70W$, flush at $1.0W$, evict $0.5W$ tokens, W = worker window.
12. Sleep-time cost (Lin et al.): $\mathrm{cost}=\mathrm{tok}_{\mathrm{sleep}}+t\cdot\mathrm{tok}_{\mathrm{test}}$, $t=10$; amortized per query cost/N over N queries sharing a context — use to decide how much idle work is worthwhile per repo.
13. Test-quality rates (Reflexion Table 2): track TP/FN/FP/TN of lesson-derived tests; FP must stay near HumanEval's 0.01, not MBPP's 0.16.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```
        type EntryKind = "fact" | "lesson" | "playbook" | "workflow" | "reflection" | "snippet";
        type Section = "strategies_and_hard_rules" | "code_snippets" | "troubleshooting" | "workflows" |
"repo_facts" | string; // ACE sections

        interface NotebookEntry {                // ACE bullet + A-MEM note + GA memory object + DC
memory_item
          id: string;                            // "<section-prefix>-<00000>", assigned by code, never by
the model (ACE)
          kind: EntryKind;
          section: Section;
          content: string;                       // ACE "content" / DC <description>
          example?: string;                      // DC <example>: code or concrete instance
          keywords: string[];                    // A-MEM K_i (>=3, most important first)
          tags: string[];                        // A-MEM G_i (domain, format, type)
          context: string;                       // A-MEM X_i: one-sentence summary
          embedding: number[];                   // A-MEM e_i = enc(concat(content, keywords, tags,
context))
          links: string[];                       // A-MEM L_i
          evidence: string[];                    // GA reflection pointers: ids of entries / trajectory ids
this was inferred from
          helpful: number; harmful: number;      // ACE counters, updated from Reflector bullet_tags
          usageCount: number;                    // DC
          importance: number;                    // GA 1..10, rated at creation
          createdAt: string; lastAccessAt: string;// GA timestamps (ISO)
          version: number; status: "active" | "retired" | "promoted";
          testIds?: string[];                    // lessons that became tests
          placeholders?: string[];               // workflows: "{product-name}" style (AWM)
          steps?: { state: string; reasoning: string; action: string }[];  // workflows (AWM)
        }

        interface Delta {                        // ACE Curator output, one per task (batch size 1)
          taskId: string; reasoning: string;
          operations: ( { type: "ADD"; section: Section; kind: EntryKind; content: string; example?: string
}
                      | { type: "UPDATE"; id: string; content: string }
                      | { type: "RETIRE"; id: string; reason: string }
                      | { type: "LINK"; from: string; to: string } )[];
        }

        interface ReflectorOutput {              // ACE Fig. 10/13 + Reflexion sr_t
          reasoning: string; error_identification: string; root_cause_analysis: string;
          correct_approach: string; key_insight: string;
          bullet_tags: { id: string; tag: "helpful" | "harmful" | "neutral" }[];
        }

        interface EvolveOutput {                 // A-MEM P_s3
          should_evolve: boolean;
          actions: ("strengthen" | "update_neighbor" | "merge" | "prune")[];
          suggested_connections: string[];
          tags_to_update: string[];
          new_context_neighborhood: string[];    // one per [new, n1..nk]
          new_tags_neighborhood: string[][];
        }

        interface CoreBlock {                    // MemGPT main context
          systemInstructions: string;            // read-only
          workingContext: string;                // fixed token budget; edited only via append/replace ops
          fifoSummary: string;                   // recursive summary of evicted turns
          pinnedEntryIds: string[];              // top playbook bullets by helpful - harmful
        }

        // On disk: notebook/entries.jsonl (append-only, one JSON per op: ADD/UPDATE/RETIRE/LINK/COUNTER —
state is a fold over the journal),
        // notebook/index.sqlite (embeddings + inverted index), notebook/core.md (rendered CoreBlock,
regenerated, never hand-merged),
        // notebook/playbook.md (rendered view: "[id] helpful=X harmful=Y :: content" grouped by section,
ACE format),
        // notebook/tests/<lesson-id>.test.ts (lesson-derived tests), recall/<session>.jsonl (raw
transcripts).
```

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Parameter	Default	Source
Delta batch size	1 task per delta	ACE §4.2
Max Reflector refinement rounds	5 (start at 2 locally)	ACE §4.2
Multi-epoch re-reflection passes (sleep)	≤ 5	ACE §4.2
Dedup cosine threshold	0.90	ACE code `--bulletpoint_analyzer_threshold`
Playbook token budget	80,000 in ACE; set to the core-block share of the worker window locally (e.g. 4–8k pinned, rest retrieved)	ACE code `playbook_token_budget`; MemGPT
Refinement mode	lazy (on budget overflow) + sleep	ACE §3
Reflexion memory bound Ω	1 for code tasks, up to 3	Reflexion §3, §4
Self-generated tests per lesson	≤ 6, AST-validated	Reflexion §4.3
Retry stop	3 consecutive failures	Reflexion §4.2
Retrieval k	10 (sweep 10–50 plateaus)	A-MEM §4 / Table 8
Neighbour candidates for link/evolve	same k = 10	A-MEM §3.2
Embedding model	all-MiniLM-L6-v2 (local, 384-d)	A-MEM
Retrieval weights $\alpha_r, \alpha_i, \alpha_v$	1, 1, 1	Generative Agents §4.1
Recency decay γ	0.995 per hour of activity	Generative Agents §4.1
Importance scale	1–10	Generative Agents §4.1
Reflection trigger	$\sum$ importance > 150 (recalibrate)	Generative Agents §4.2
Reflection inputs	100 most recent notes → 3 questions → 5 insights each	Generative Agents §4.2
Memory-pressure warn / flush / evict	0.70 W / 1.0 W / 0.5 W	MemGPT §2
DC curated memory length	≈2,000–2,500 words per section view	Dynamic Cheatsheet appendix
DC-RS retrieval k	3 past I/O pairs	Dynamic Cheatsheet §2
Sleep-time parallel rethinks	$k = 5$ (beats 10)	Sleep-time compute §4
Sleep/test cost weight t	10 (reinterpret as idle vs interactive)	Sleep-time compute §2
Workflow induction source	verified-successful trajectories only	AWM §5 (noisy evaluator warning)
LLM temperature for induction / agent	0.0	AWM §3

2.5 Evaluation: how to know the module works (metrics, from the papers)

- **Task success with vs without the Notebook** on a held-out rotation of the user's own past tasks (ACE's offline/online protocol): report pass@1 of the verifier cascade; expect the ACE pattern — gains concentrated on harder tasks (test-challenge +24.5 TGC online) and gains that vanish or invert when the feedback signal is removed (finance no-GT rows).
- **Context-collapse guard**: log `tokens(notebook)` and `tokens(core)` per consolidation; any single-step drop > 50% without an explicit RETIRE batch is a failure (ACE 18,282 → 122 case).
- **Retrieval quality** on an internal LoCoMo/DMR-style probe: generate QA pairs from past sessions (MemGPT DMR; A-MEM categories single-hop, multi-hop, temporal, open-domain, adversarial); measure F1 / BLEU-1 and tokens per answer; target the A-MEM profile (≤ 2.5k tokens per answer vs full-history ≈17k).
- **Counter sanity**: fraction of retrieved bullets tagged helpful vs harmful per task; a healthy playbook has rising helpful/harmful ratio and shrinking harmful-retire churn; AWM's 18.5% workflow-utilization figure is the floor to beat for `workflow` entries.
- **Lesson tests**: TP/FP/FN/TN of lesson-derived tests against pre-fix and post-fix snapshots (Reflexion Table 2); FP ≤ 0.05.
- **Sleep efficiency**: test-time tokens to reach the same pass rate with vs without the pre-computed `c'` (Sleep-time compute's ≈5× at low budgets; expect ≈1.5× on SWE-like tasks) and per-query amortized

sleep cost.
- **Ablations to run once**: no Reflector (ACE −1.7 avg), no multi-epoch (−2.6), no offline warmup (−3.4), no link/evolve (A-MEM −17 F1 multi-hop), no reflection (GA −3 TrueSkill), full-rewrite curator instead of deltas (expect collapse).

2.6 Known pitfalls and failure modes (from the papers' limitations)

- **Context collapse** from any LLM full rewrite of the memory (ACE §2.2; DC's curator must copy content forward and models abbreviate "Previous content preserved"). Only deltas; append-only journal.
- **Brevity bias**: an optimizer that scores shorter prompts higher will strip domain detail (ACE; Gao et al.). Do not reward brevity in the Curator; cap by budget instead.
- **Degradation without a feedback signal**: ACE and DC both fall below baseline on FiNER with no labels; the Curator must receive verifier outcomes, and unverified tasks should produce `reflection` entries only, not `playbook` entries.
- **Weak curator**: small models curate badly (DC Haiku/4o-mini; ACE "reliance on a strong Reflector"; MemGPT GPT-3.5 function-calling failures). Reflector/Curator/evolve on the senior model, never the 9B worker.
- **False-positive tests** dominate the damage in test-driven loops (Reflexion MBPP FP 0.16 → net loss). Keep a lesson test only if it fails before the fix.
- **Noisy online induction**: workflows induced from wrongly-judged successes hurt in-domain (AWM); low workflow usage (18.5%) and divergence errors when following stale workflows; environments change (AWM pop-ups) → attach repo commit hash to `workflow` and `fact` entries and re-validate on drift.
- **Retrieval misses and embellishment** as memory grows (Generative Agents §7.2); fragments retrieved without their context → always return linked neighbours and the `context` sentence, and cite evidence ids so the worker can distinguish recalled from inferred.
- **Local minima** of verbal reinforcement (Reflexion WebShop): after Ω reflections with no progress, escalate to the Thinking tier instead of reflecting again.
- **Early termination of paged search** (MemGPT) → enumerate pages deterministically.
- **Sleep-time over-editing** (SWE-Features: more files touched, lower precision at high budgets) and **unpredictable queries** → sleep output is advisory, stored as `fact` with provenance, never auto-applied.
- **Memory hacking / prompt injection via stored text** (Generative Agents §8.2): treat Notebook content retrieved from logs of external tool output as data; the Curator should not ADD imperative text from tool outputs.

2.7 Open questions this section does not settle

- Exact token split of the core block (system / working context / pinned bullets / summary) for a 9B worker with a ~16–32k window; MemGPT and ACE give fractions and an 80k playbook budget but not a small-window split.
- Recalibration of the Generative-Agents reflection threshold (150) and decay unit (game hours) to a coding workload; whether `importance` should be replaced by verifier-derived signals (test count affected, time saved).
- Whether the Reflector's helpful/harmful tags from a 9B Generator's `bullet_ids` are reliable enough, or whether usage must be inferred from the trajectory by the senior.
- How many consolidation epochs over the same week's trajectories are useful locally before over-fitting the playbook to recent tasks (ACE used ≤ 5 on benchmark splits).
- Promotion criteria from `lesson` → `playbook` → Skill-tier code (Section B): the papers give counters but no threshold; start with helpful ≥ 3, harmful = 0, and a passing test.
- Cross-model memory transfer: DC reports "mixed results" when memory written by a large model is read by a small one; whether senior-written bullets need a worker-readability pass is untested.
- The split of idle compute between consolidation (merge/decay/promote), reflection and repo pre-computation; Sleep-time compute leaves the sleep/test allocation open.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2510.04618	Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models	2025 (v3 2026)	ICLR 2026	https://github.com/ace-agent/ace	https://arxiv.org/pdf/2510.04618
2303.11366	Reflexion: Language Agents with Verbal Reinforcement Learning	2023	NeurIPS 2023 (not stated on arXiv page)	https://github.com/noahshinn024/reflexion	https://arxiv.org/pdf/2303.11366
2504.07952	Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory	2025	arXiv preprint	https://github.com/suzgunmirac/dynamic-cheatsheet	https://arxiv.org/pdf/2504.07952
2502.12110	A-MEM: Agentic Memory for LLM Agents	2025	NeurIPS 2025	https://github.com/WujiangXu/A-mem-sys	https://arxiv.org/pdf/2502.12110
2409.07429	Agent Workflow Memory	2024	ICML 2025 (PMLR v267)	https://github.com/zorazrw/agent-workflow-memory	https://arxiv.org/pdf/2409.07429
2304.03442	Generative Agents: Interactive Simulacra of Human Behavior	2023	UIST 2023 (not stated on arXiv page)	https://github.com/joonspk-research/generative_agents	https://arxiv.org/pdf/2304.03442
2310.08560	MemGPT: Towards LLMs as Operating Systems	2023 (v2 2024)	arXiv preprint	https://research.memgpt.ai	https://arxiv.org/pdf/2310.08560
2504.13171	Sleep-time Compute: Beyond Inference Scaling at Test-time	2025	arXiv preprint	https://github.com/letta-ai/sleep-time-compute	https://arxiv.org/pdf/2504.13171